

Vericoding with Axiomander

Specification is the control surface.

Code is a derived artifact.

Correctness requires proof.

The optimism problem

LLMs are **trained to satisfy the prompt**, not to be correct.

- They produce code that *looks* like a solution to the request.
- They are systematically **optimistic**: edge cases, error paths, and invariants get glossed.
- "It compiles and the happy path runs" is treated as success.
- The failure mode is silent: plausible code that is subtly wrong.

The model optimises for **apparent** intent satisfaction. We need a way to test **actual** intent satisfaction — **deterministically**.

The ladder of guarantees

How strongly can we pin down intent?

Mechanism	Catches	Misses
Prompt review	gross misunderstanding	everything subtle
Types	shape errors	values, relations, invariants
Tests	the cases you thought of	the cases you didn't
Strong specification	anything expressible as a property	only what you leave unstated

Types and tests are *samples* of intent. A **strong specification** makes intent **explicit and mechanically checkable** over *all* inputs — and it can be **proved**, not sampled.

Tests sample. Proofs quantify.

A test

```
def test_clamp():  
    assert clamp(5, 0, 10) == 5  
    assert clamp(-3, 0, 10) == 0  
    assert clamp(99, 0, 10) == 10
```

A finite set of points. Green means *these three* are right.

A specification

$\forall v, lo, hi. lo \leq hi \Rightarrow lo \leq \text{clamp}(v, lo, hi) \leq hi$

A statement over **all** inputs. Proved means *every* input is right.

A passing test is evidence. **A proof is a guarantee.**

...a guarantee **relative to the specification**. Proof establishes correctness w.r.t. the spec, not w.r.t. unstated intent — so the spec itself becomes the artifact we must get right.

The spec is now the thing to get right

Proof moves the burden, it doesn't remove it. Correctness is **relative to the specification**, so a spec can still:

- omit a requirement, or encode a wrong assumption;
- over-constrain (reject valid behaviour) or under-constrain (allow wrong behaviour).

This is a feature of *where the work goes*, not a hole. The reviewable surface shrinks from "all the code" to "the contracts" — and the contracts are small, declarative, and the explicit object of review.

What "vericoding" means

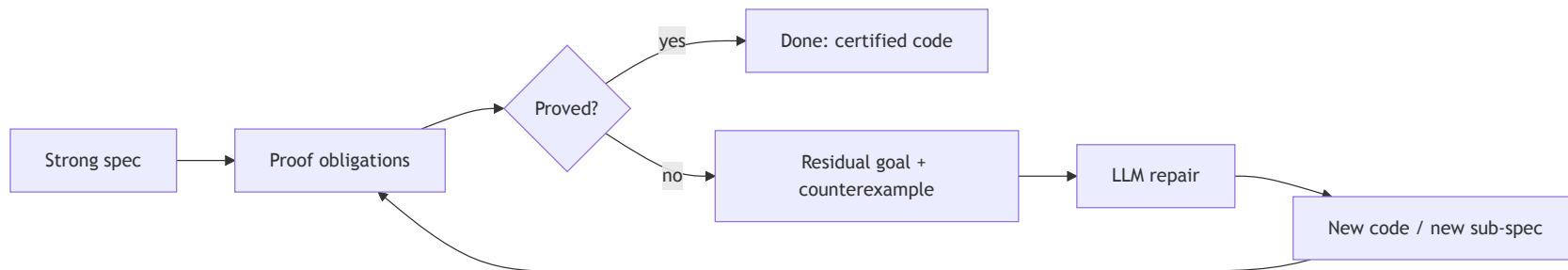
A development loop in which:

1. The human writes and iterates on a **strong specification**.
2. An LLM proposes an **implementation** to satisfy it.
3. A **verifier** decides — deterministically — whether the code meets the spec.
4. On failure, the verifier returns a **structured residual** that drives the next attempt.

The spec is the thing you maintain. The code is generated, checked, and regenerated underneath it.

The heal-loop

The verifier doesn't just say "no" — it says **why**, in a form the LLM can act on.



- A counterexample is a **concrete witness** of failure — typed values, not vibes.
- A residual goal is the **exact remaining obligation** with its hypotheses.
- The LLM repairs against the residual, not the original prompt.

Why the heal-loop beats re-prompting

Naive: "try again"

```
LLM: here is code
human: it's wrong
LLM: here is different code
human: still wrong
...
```

No signal. The model guesses again.

Heal-loop: drive on residual

```
Goal:
  lo <= result <= hi
Hyp:
  val > hi
Counterexample:
  result = val  (= 99, > hi)
```

The model gets the **failing fact**. It fixes *that*.

Determinism turns "convince a reviewer" into "close a goal."

Composability enables scaling

If every edit re-checked the whole program, vericoding would not scale.

- **Iteration cost.** You change one spec; you should re-verify *only* what depends on it.
- **Stubbing.** A library you can't verify is replaced by its **contract**. Callers prove against the stub's axioms.
- **Locality of trust.** A function's body can change freely as long as its contract is stable — callers are untouched.

Body changes

invalidate local proofs

Contract changes

invalidate callers

The argument in one slide

- LLMs are **optimistic**; they satisfy the prompt, not the intent.
- Types and tests **sample** intent; a strong specification makes it **explicit and checkable**.
- Make the **specification the control surface** — iterate on it, derive code under it.
- A **heal-loop** drives the LLM on residual goals and counterexamples, not re-prompts.
- The LLM has freedom in **sub-specification**; helpers get contracts, checked the same way.
- **Composability** (frame rule + contract hashing) makes iteration and **stubbing** scale.
- The Axiomander stack — **LLM + Rocq + Iris + Python** — reduces all of this to soundly-checked proof obligations.

Specification is the control surface. Code is a derived artifact. Correctness requires proof.

layout: center class: text-center

Thank you

Axiomander 

Iterated specification management for Python.

github.com/scidonia/axiomander

See the whitepaper: docs/whitepaper.md